CMSC 35200

Deep Learning Systems

Rick Stevens

Argonne National Laboratory
The University of Chicago

2026 introduction · DRAFT · 27 September 2026

The 2026 course

Understand the models and the systems that make them useful.

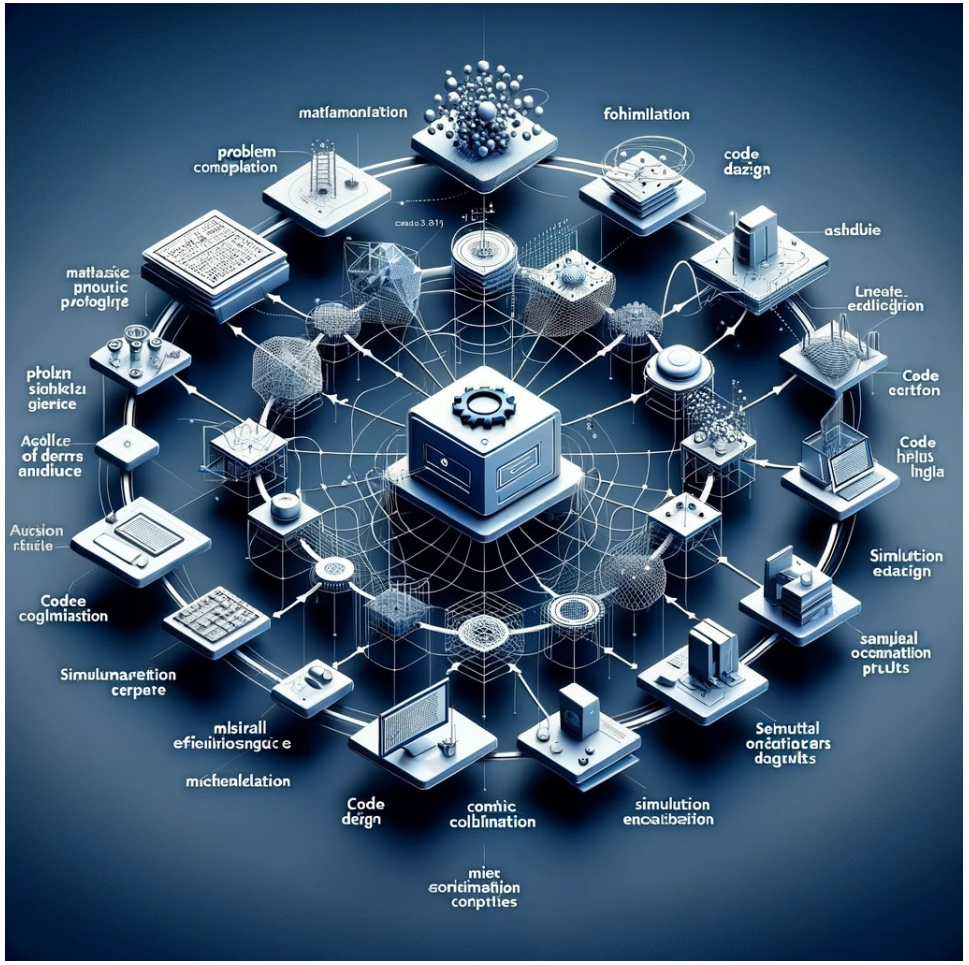
Build with coding agents, then verify the result independently.

Measure accuracy, latency, memory, concurrency and cost.

Use scientific problems to test the limits of scalable cognition.

AI Assistant for “Computational” Science

- Open Problem
- Mathematical Formulation
- Numerical or Symbolic Formulation
- Efficient Algorithm
- Parallel HPC implementation
- Simulation Campaign at Exascale HPC
- Analysis of Output
- Insights and Closing the Loop



GPT4 Notion of what AI assistant Looks like

Research ambitions and testable claims

Scientific discovery motivates the systems we will build.

A compelling demonstration needs a falsifiable claim and a baseline.

The next slides describe a research agenda.

Our projects will test a bounded part of that agenda.

What it might take to get to a **“scientific superintelligence”** that can run for days or weeks without intervention, master the literature, design instruments, direct large robotic facilities, and originate new theories.

The scientific discovery cycle

Implementation synthesis based on DOE’s closed-loop vision

01 **Frame the question**

Set the scientific objective, constraints and success criteria.

02 **Assemble evidence**

Retrieve relevant observations, theory and prior experiments.

03 **Propose and predict**

Generate hypotheses and estimate which tests will teach the most.

04 **Run the test**

Execute approved simulations or physical experiments.

05 **Validate the result**

Compare prediction with measurement and quantify uncertainty.

06 **Update and repeat**

Preserve the evidence and use it to choose the next question.

Synthesis informed by DOE National Challenges and Models teams

A portfolio of scientific models

Different scientific tasks need different model capabilities.

Reasoning models

Interpret questions, assemble evidence and plan scientific work.

Domain foundation models

Represent scientific structures and relationships in specialized data.

Surrogates and predictors

Approximate costly calculations and estimate promising candidates.

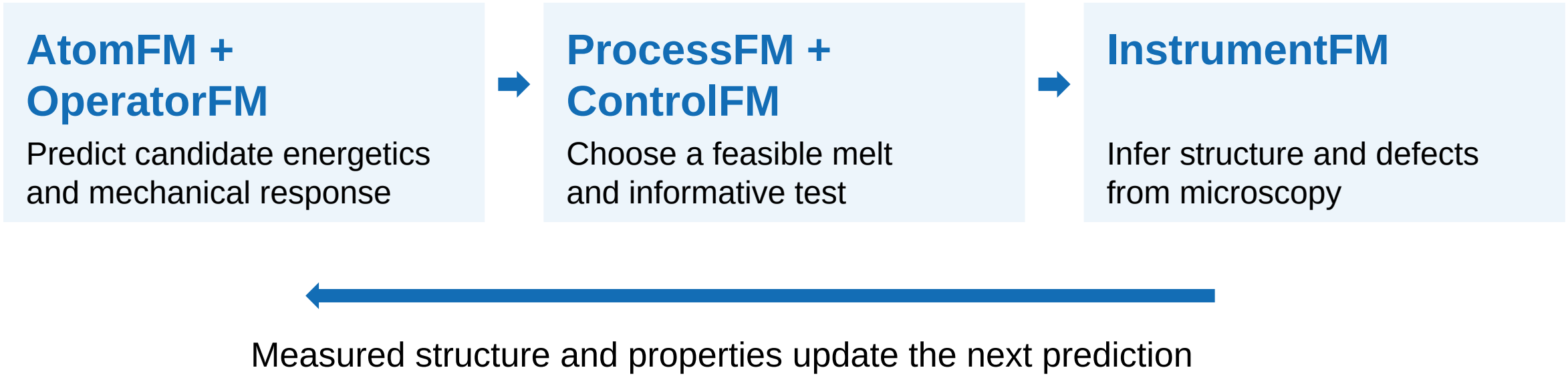
Evaluation and model registry

Track capabilities, limitations, versions and evidence for reuse.

Open-weight scientific models add a path for community adaptation and scrutiny.

A discovery requires models to compose

Question: which oxygen-containing alloy should we make next, and why?



CodeFM executes simulations. EvidenceFM records the chain of justification.

Course roadmap

Weeks 1–2 Coding loops, tensor notation and transformer foundations

Weeks 3–4 Attention variants, multimodal models and scaling

Weeks 5–6 Inference systems, concurrency and resource planning

Weeks 7–8 Agent evaluation, decision models and scientific computing

Weeks 9–10 Scalable cognition, replication and project defenses

Key Transformer Paper

arXiv:1706.03762v5 [cs.CL] 6 Dec 2017

Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com

Noam Shazeer* Google Brain noam@google.com

Niki Parmar* Google Research nikip@google.com

Jakob Uszkoreit* Google Research usz@google.com

Llion Jones* Google Research llion@google.com

Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu

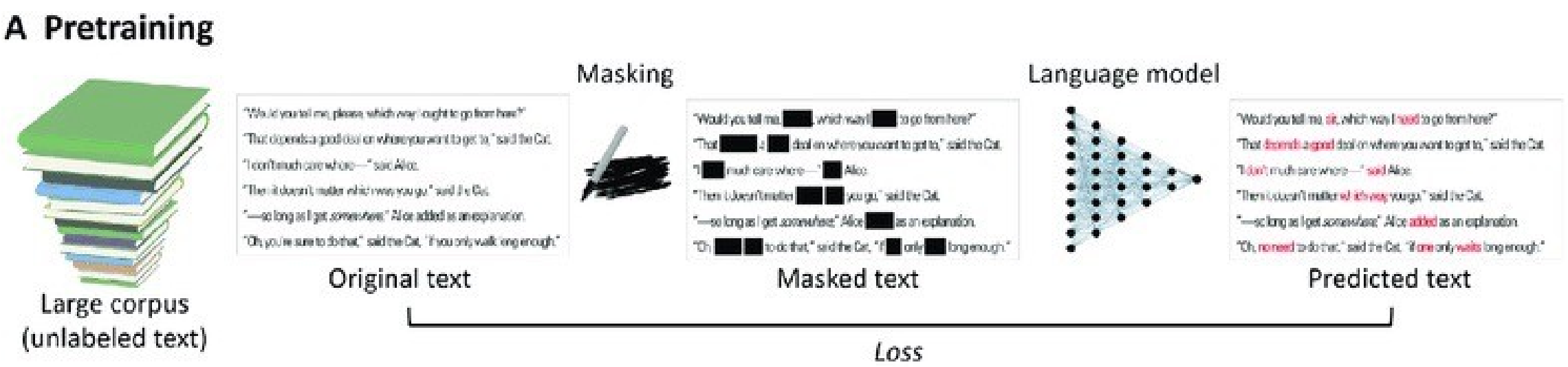
Łukasz Kaiser* Google Brain lukaszkaiser@google.com

Illia Polosukhin* ‡ illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

Self-Supervised Pretraining and Fine-Tuning



Tensor notation as an executable specification

$$Q_{bthk} = X_{bti}\ W_{ihk}$$

$$S_{bhts} = Q_{bthk}\ K_{bshk}\ /\ \sqrt{D}$$

$$A_{bhts} = \mathrm{softmax}_s(S_{bhts} + M_{ts})$$

$$O_{bthv} = A_{bhts}\ V_{bshv}$$

Sum contracted indices i, k and s. D is the key width.
Free indices preserve batch, position and head structure.

Attention variants and memory

Multi-head attention provides the reference implementation.

GQA shares key/value heads across groups of query heads.

MLA compresses the representation retained in the KV cache.

Linear and gated recurrent attention change the state and computation.

LLM and multimodal scaling

Training scale includes parameters, tokens and data quality.

Inference scale includes context, sampling and reasoning budgets.

Images, video and scientific arrays add representation and alignment costs.

The experiment must report capability together with memory and time.

Jev as a decision-model case study

Jev exposes typed decisions rather than free-form answer generation.

Compare decision quality and calibration with a language-model baseline.

Measure latency, abstention and the cost of a wrong decision.

Valid output structure still requires a correct decision.

Agents turn capabilities into scientific work

The research task is a persistent stateful campaign, not a sequence of isolated prompts.

Reason and plan

Track hypotheses, competing explanations and uncertainty; choose the next simulation or experiment.

Evidence and simulator grounding

Execute and recover

Call domain models, schedule ensembles, use instruments and recover from routine failures.

Typed tools and durable task state

Verify and govern

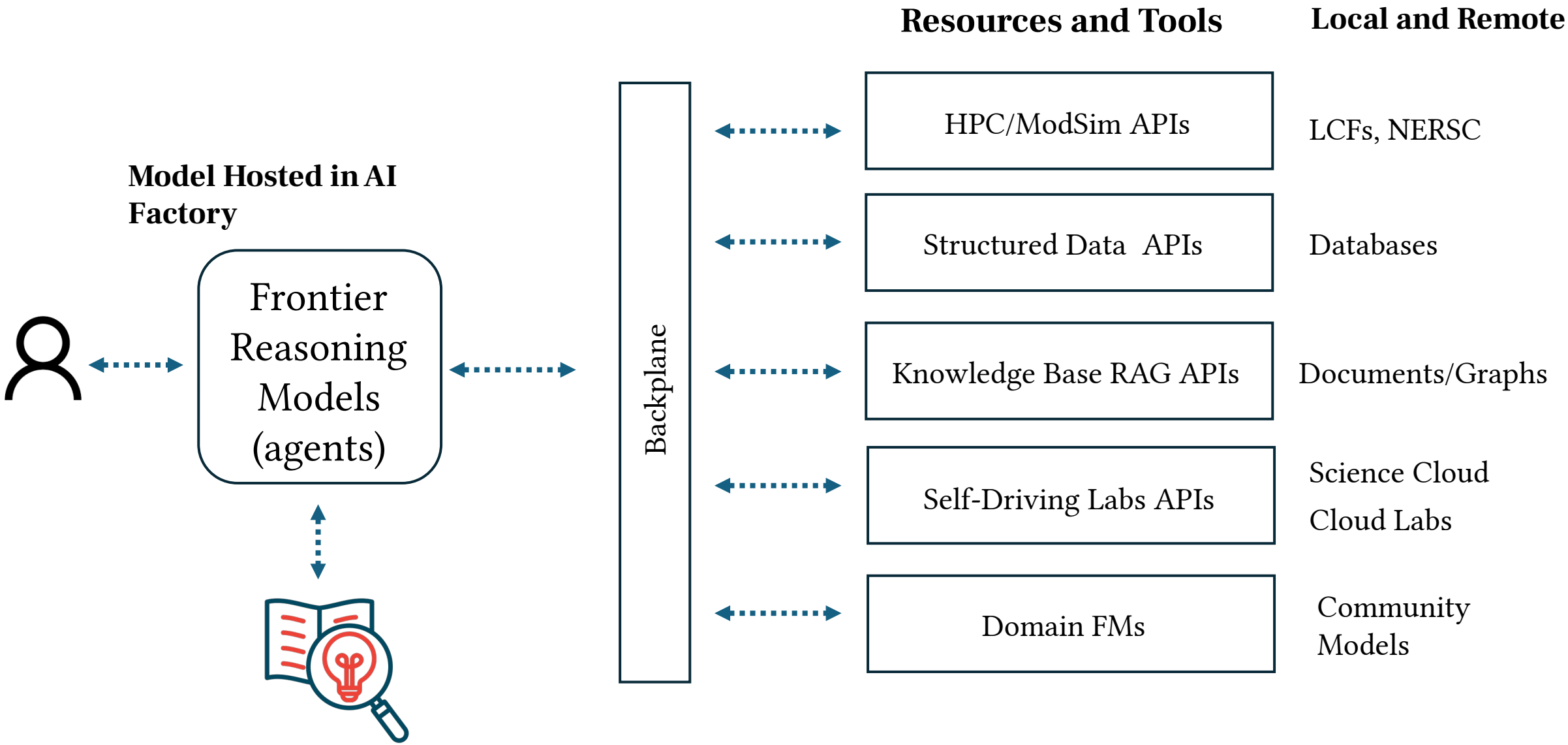
Check physical constraints, preserve provenance and route consequential actions through defined authority.

Runtime monitoring and escalation

Multi-day autonomy requires state, budgets, boundaries and recovery.

Source: ModCon notes + projects 297659 and 301923 • Synthesis

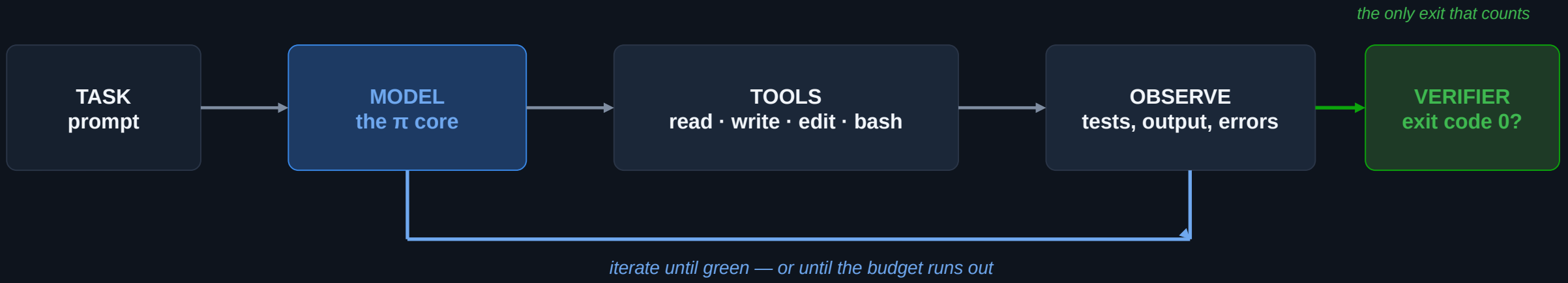
Scientific AI Platform Example



THE BIG IDEA

Models aren't chatbots here — they're loop cores

An agent loop: the model reads a task, acts through tools, observes reality, and iterates until an independent judge says done.



file-path discipline

artifacts must land where the grader looks

recover from red

read a pytest traceback, make the smallest fix

drive multi-step tools

no wandering, no giving up early

know when it's done

stop on green — don't 'improve' passing code

A working coding loop on every laptop

A reproducible Python environment and a clean Git checkout

A coding agent that can read, edit and run the project tests

One approved model endpoint and an offline test path

A failing seed, a verified repair and a recorded run

THE BENCHMARK

pi-30 in one slide

1 problem = seed code (pre-verified broken) + task prompt + loop budget + **verifier (exit code = verdict)**

30 problems

P1–P30, authored as a curriculum: write-from-spec → fix-failing-tests → numerical → optimize → tournaments & critics.

1 harness

run_model_30.sh — problem statements embedded, every pi call timeout-wrapped, resume-safe, fleet-parallelizable.

1 lane = model × backend

The same weights on two serving stacks are two lanes. 26 unique lanes have completed the sweep.

1 verdict per problem

Binary PASS/FAIL from an independent grader the model may not edit. Score = passes out of 30. No partial credit, no vibes.

STANDING RULE 1 / 6

The verifier is the only source of truth

- Model prose is never trusted for scoring. Not “done!”, not “all tests pass”, not a beautiful summary.
- Each problem ships an independent grader: verify.py, check.py, pytest, a benchmark scorer, or a convergence check.
- The grader's process exit code is the verdict: 0 = PASS, anything else = FAIL.
- The model is forbidden from editing the grader — every prompt says so, and staging restores pristine copies each run.

run_model_30.sh + p1/verify.py

```
# harness verdict line — the whole religion:
(cd "$ROOT/p1" && "$PY" verify.py) \
  >/dev/null 2>&1 \
  && record "P1: PASS" \
  || record "P1: FAIL"

# verify.py ends with hard asserts:
assert rc == 0
assert abs(d["stdev"] - 2.138089935) < 1e-6
print("OK P1")   # exit 0 = the verdict
```

Why so strict? agent models are trained to sound finished. An eval that reads prose measures persuasion; an eval that runs code measures capability. (You'll see models claim victory over code that doesn't import.)

ANATOMY

Inside one problem directory

problems/p24 (token bucket)

```
problems/p24/
├─ limiter.py      ← SEED (broken on
│                     purpose; must fail)
└─ verify.py       ← GRADER (never
                     edited by the model)

# graders across the suite:
verify.py          behavior asserts
check.py           oracle diff
test_thing.py      pytest suites
bench.py + check.py  speed + correctness
score.py           tournament scorer
converge_check.py  numerical convergence
PLAN.md / CRITIQUE.md  fan-out & critic aids
```

The staging contract (per lane)

- rm -rf runs30/<tag> — a lane never sees a previous lane's artifacts
- Fresh copies of every verifier + seed staged into runs30/<tag>/pN/
- pi runs with cwd = the problem dir; the grader runs from the same dir moments later
- Verifiers execute under a pinned Python (numpy + pytest available) — not whatever the model's bash found on PATH
- “DO NOT edit check.py/verify.py” appears in every prompt — and staging would clobber the edit anyway

Backend access is part of the system

Record provider, endpoint, model ID, version and serving configuration.

Check authentication, tool calls, timeouts and rate limits before grading.

Set token and spending budgets. Keep credentials outside the repository.

Separate infrastructure failures from task failures.

The benchmark contract

Freeze tasks, held-out tests, prompts and resource budgets.

Compare systems on the same tasks with independent repeats.

Report success rate, failure classes, latency and cost per valid result.

Keep raw artifacts and distinguish first attempts from retries.

Concurrency changes the experiment

Independent trials can run in parallel. Shared state introduces contention.

Queues, retries and cancellation affect measured throughput.

More agents can duplicate work and amplify correlated errors.

Compare serial and concurrent runs at a fixed total budget.

SCALE OUT

Fleet mode: 13 lanes in parallel, politely

harness/fleet_launch.sh (condensed)

```
MODELS=(gemma4-e2b gemma4-e4b gemma4-12b
  gemma4-31b ornith-9b devstral-small-2
  qwen36-27b uic-ornith-9b … )  # 13 total

for m in "${MODELS[@]}"; do
  PI30_RESUME=1 PI_TIMEOUT=360 \
  setsid bash run_model_30.sh \
    "$PROVIDER" "$m" "fleet-$m-$STAMP" \
    > "$log" 2>&1 &
  setsid bash -c "… heartbeat …" &
done; wait

# setsid: each lane gets its OWN process group —
# SIGTERM to the launcher cascade-kills NOTHING
```

Fleet furniture

- One aggregator provider (chiatta00-agg) fronts all 13 models — the launcher just varies --model
- Heartbeat sidecar per lane: greps the latest [Pn] marker + pass/fail counts into <model>.cur every 5 s
- PI30_RESUME=1 by default — a re-launch continues every partial lane instead of burning finished work

Architecture and resource planning

Estimate weight storage, KV state, activations and runtime overhead.

Separate prefill, decoding and tool execution in the time model.

Plan training, serving and agent experiments as different workloads.

Use measured utilization and a sensitivity analysis.

The hypothetical 10T MoE workload

Base case: 10T total parameters, 500B active per token, 20T training tokens.

Quantity	Planning assumption	Implication
Training duration and precision	100 days, FP8 mixed precision	BF16/FP32 state retained
Training model state	16 bytes/parameter + 25% reserve	200 TB sharded memory floor
Inference weights	FP4 + 10% quantization metadata	5.5 TB resident weights
Inference context and batch	8,192 input tokens, 64 sequences	137 GB KV cache at assumed layout
Output and service demand	1,024 output tokens, 10K output tok/s	Separate decode and prefill energy

These are analyst assumptions. 20T training tokens is a scenario, not an assertion of optimal model quality.

A small planning exercise

Illustration: 20 tasks × 3 repeats = 60 trials

At 8,000 input + 2,000 output tokens per trial: 600,000 tokens

Price input and output separately using the approved endpoint rate.

Measure one trial before selecting concurrency or committing the budget.

A theory of scalable cognition

Working question: when does additional computation improve verified work?

Vary model capability, memory, tools, search depth and parallel effort.

Track useful progress per unit of time, tokens and energy when measurable.

Test limits from coordination, correlated errors and weak verification.

The IQ 150 thought piece

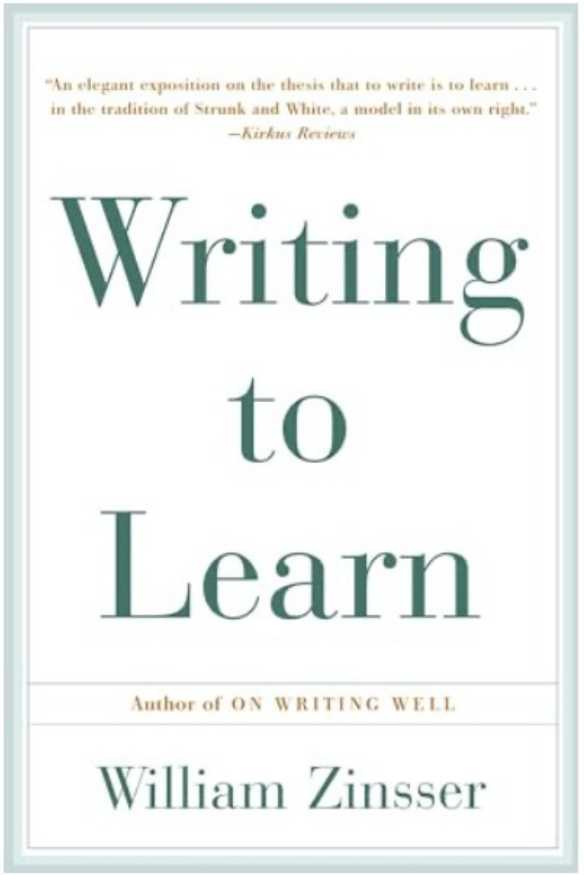
What scientific task would this label help us predict?

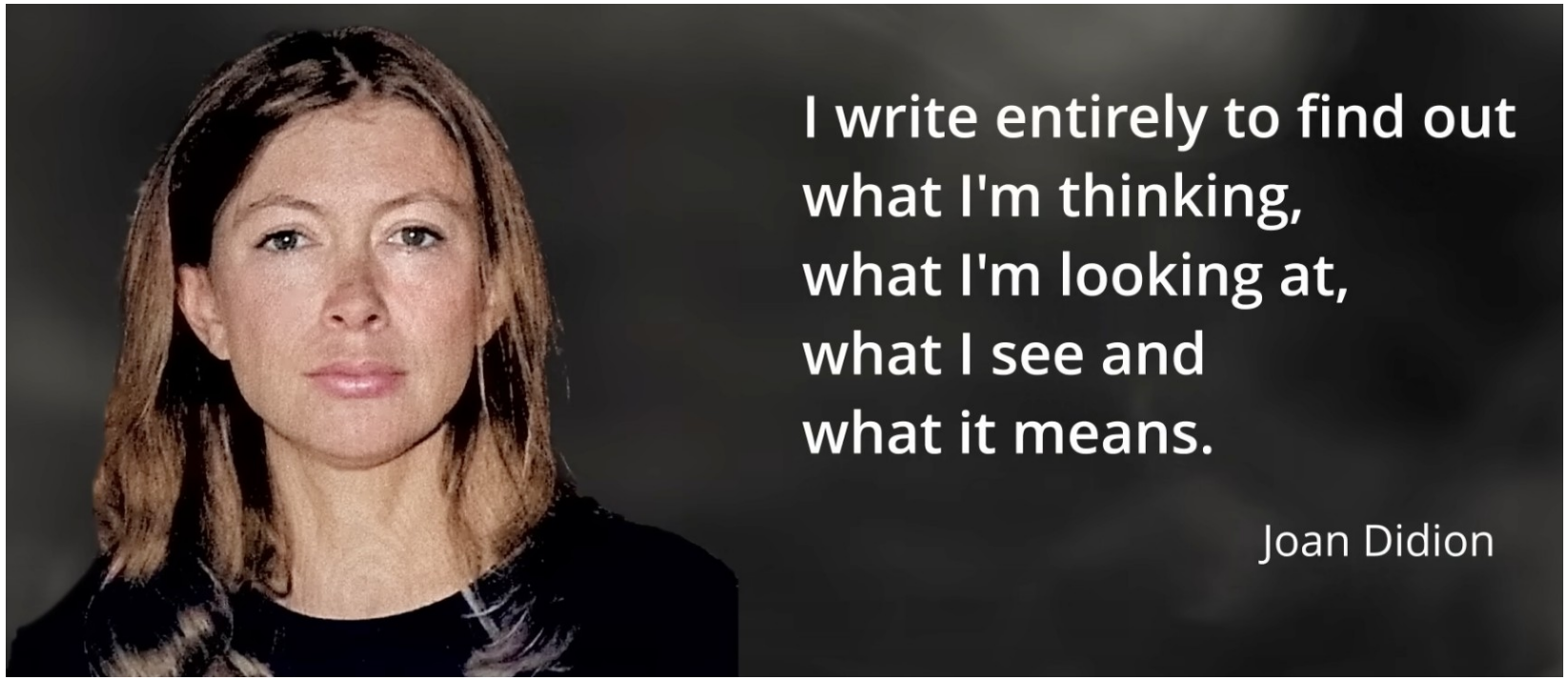
How do test exposure, tools and time budgets affect a score?

Which failures remain invisible to a single aggregate number?

Propose a capability profile and an experiment that could refute it.

I strongly encourage you to journal this class





AI review of the course materials

Audit every lecture, note, paper link, assignment and rubric.

Check factual claims, mathematical notation and executable examples.

Record source, version, issue, proposed correction and reviewer.

An instructor resolves uncertain claims and approves material for release.

A repository for students and agents

Each module includes lecture notes and curated background papers.

Assignments ship code, tests, run instructions and evaluation rubrics.

Thought pieces connect assumptions to evidence and counterexamples.

Challenge records state the question, baseline, budget and success test.

Six labs and a project

Lab 0 Coding-loop setup and backend smoke tests

Labs 1–2 Einstein notation, attention variants and multimodal scaling

Labs 3–4 Serving, concurrency, benchmarking and Jev decisions

Lab 5 Scientific replication and a bounded Navier–Stokes task

Assessment with agents available

50% labs Correctness, experiments and reproducible evidence

40% project Question, implementation, validation and oral defense

10% participation Thought pieces, reviews and the learning journal

Students explain and defend their own submitted work.

The ATLAS of problems

Start with the reviewed AI ATLAS edition dated 18 September 2026.

100 entries cover scientific questions, methods and verification projects.

Choose a tractable subproblem with data and an independent success test.

Keep the original scientific ambition and state the limits of the result.

Navier–Stokes as a bounded challenge

Define the PDE, domain, boundary conditions and initial data.

Compare a numerical baseline with a learned surrogate.

Evaluate held-out trajectories, rollout stability and physical constraints.

Separate numerical error, modeling error and distribution shift.

Models that are facile with theory and can create and debate alternative mechanisms

3. **Theory formation and mechanistic explanation**

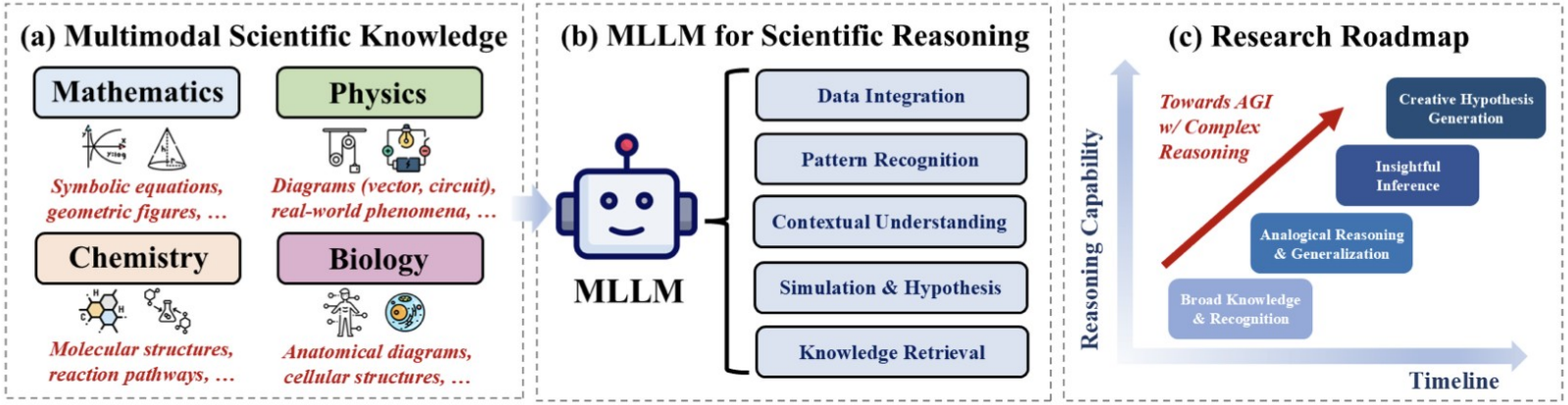
- **What:** Integrations of neural reasoning, program synthesis, and formal methods that propose *explanatory* models (laws, symmetries, invariants), not just predictive fits.
- **Why:** Scientific value is explanation that compresses data and predicts out-of-distribution.
- **Good looks like:** Auto-generated, **machine-checkable** theories that (i) pass formal proof checks where applicable and (ii) make novel, confirmed predictions. Progress in autoformalization/theorem-proving is an existence proof of the tooling direction. arXiv +2

Combining current state of the art reasoning models with theorem provers (Lean4, Coq, etc) and symbolic computing engines (sympy sandbox) gets us part way to this goal. Specialized models that are trained to leverage these tools might do even better in the near term.

Multimodal scientific reading and reasoning models that understand provenance and time

4. **Multimodal scientific reading with provenance**

- **What:** Robust ingestion of papers, figures, tables, methods, code, and data; automatic replication; citation-level provenance and uncertainty.
- **Why:** Mastery of literature must be *verifiable* and *faithful*, not a paraphrase generator.
- **Good looks like:** Reproduce ≥80% of computational results for a representative set of papers (code+data), generate discrepancy reports with exact citations, and attach confidence to every claim.



Thought pieces throughout the course

When does a verifier deserve trust?

What does a faster or larger system actually explain?

Can agent memory and concurrency produce reliable cumulative progress?

What evidence would distinguish discovery from plausible synthesis?

The project evidence package

A precise question and a budgeted experimental plan

A versioned implementation with baselines and ablations

Data provenance, raw results, failure analysis and reproducible commands

A short paper, poster and oral defense

Outcomes we are driving towards

- **An AI that proposes a theory**
  not just a fit to data, but a falsifiable mechanism that predicts new phenomena and survives experimental test.

- **An AI that designs and runs its own experiments**
  closing the loop from hypothesis to instrument control to result interpretation to next experiment.

- **An AI that reliably says 'I don't know'**
  with calibrated uncertainty that scientists trust enough to act on — including knowing when not to act.

- **Scientific AI as public infrastructure**
  open models, shared compute, common standards — so every lab in every country can participate.

- **Incentive reform**
  where replication, negative results, data curation, and maintenance are valued as highly as novel discoveries.

The first week

Read the course guide and create a learning journal.

Complete the environment and backend checks.

Run a deliberately failing example, repair it with an agent and verify it.

Submit the patch, test evidence and a short account of what failed.

Questions that will guide the quarter

Which capabilities can we verify independently?

How does systems design change the capability we observe?

Where does additional scale stop helping?

What would count as a scientific contribution?

REFERENCE

Selected sources

2025 foundation LS-2025-First-Lecture

Scientific systems Purdue September 2026 and Scientific Superintelligence

Coding loops Closing the Loop pi-30 tutorial, July 2026

Planning and challenges CODE-A-THON-RICK and reviewed AI ATLAS